



**POLYTECHNIQUE
MONTRÉAL**

UNIVERSITÉ
D'INGÉNIERIE

Département de génie informatique et génie logiciel

Cours LOG3430:

Méthodes de test et de validation du logiciel

TP5 - Tests de charge

Groupe 03

Noms et matricules :

Aiya Ben Ouhida (2283873)

Arman Lidder (2174916)

Soumis à :

Samin Niazi

Date de remise:

Le 23 novembre 2025

Partie 1:

Question 1.1

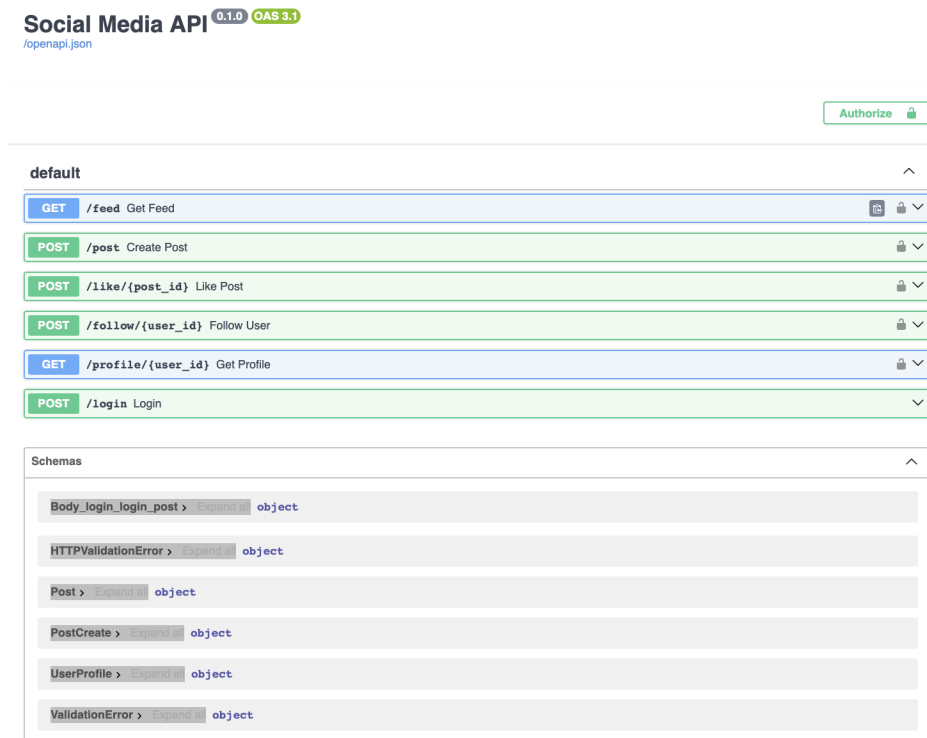


Figure 1 - Page /docs du serveur

Question 1.2



Host

http://127.0.0.1:8080

Status

RUNNING

Users

10

RPS

3.4

Failures

0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Login	10	0	3	4	4	2.74	2	4	55.5	0	0
GET	View Feed	689	0	1	2	2	1.39	1	9	754.88	3.4	0
	Aggregated	699	0	1	2	3	1.41	1	9	744.88	3.4	0



Figure 2 - Résultat de l'exécution de locustfile.py avec 10 utilisateurs

Partie 2:

Question 2.1

Le script d'extraction a été généré à l'aide de ChatGPT. Pour ce faire, nous avons copié une portion du fichier logs.txt incluant les différentes méthodes HTTP présentes, puis demandé la génération d'un script capable de lire le fichier et d'en extraire les informations utiles aux tests de charge. Ce script nous permet notamment d'obtenir le nombre de requêtes par endpoint, information nécessaire pour la modélisation de la charge globale.

Voici le résultat du script:

```

=== Charge globale (nombre de requêtes et ratios) ===
/feed                353      (34.54%)
/like/:id            235      (22.99%)
/profile/:id         223      (21.82%)
/follow/:id          149      (14.58%)
/post                52       (5.09%)
/login               10       (0.98%)

```

Figure 3 - Résultat de l'exécution du script pour charge globale

Question 2.2

```

from locust import task, between, TaskSet, User, HttpUser
from extract_logs import endpoint_counts
from client import SocialMediaClient
import random

ENDPOINTS = {
    "/feed" : "view_feed",
    "/like/:id" : "like_post",
    "/profile/:id" : "view_profile",
    "/follow/:id" : "follow_user",
    "/post" : "create_post"
}

class GlobalUser(TaskSet):
    """usage new"""
    def on_start(self):
        """new"""
        self.social_media_client = SocialMediaClient(self.client)

        total = 0
        self.dict = {}
        for endpoint in ENDPOINTS:
            if endpoint in endpoint_counts:
                total += endpoint_counts[endpoint]

        for endpoint in ENDPOINTS:
            if endpoint in endpoint_counts:
                self.dict[endpoint] = endpoint_counts[endpoint]/total

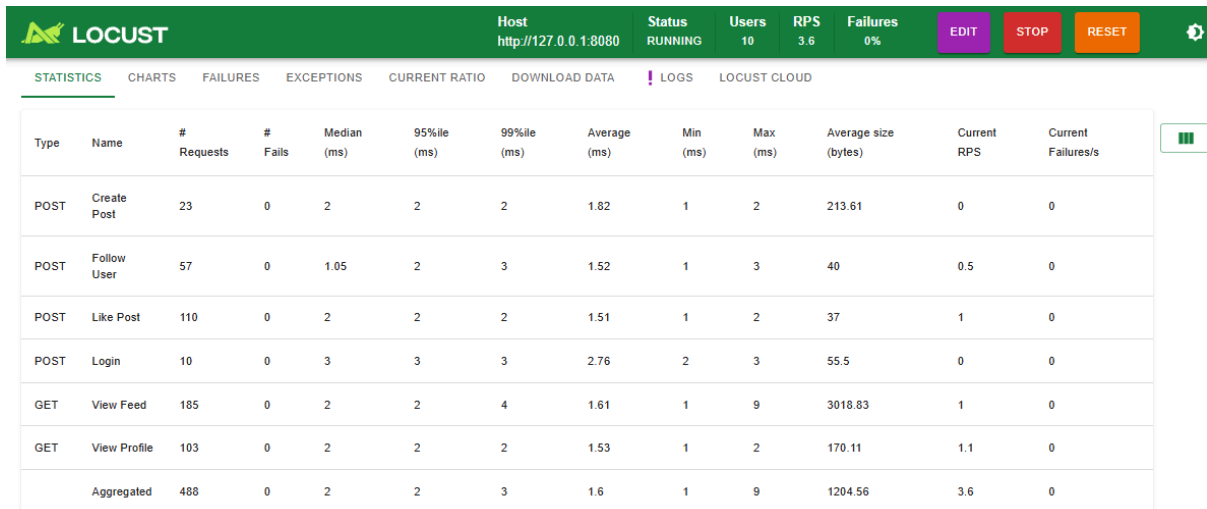
    @task
    def perform_api(self):
        endpoint = random.choices(list(self.dict.keys()), weights=list(self.dict.values()))[0]
        method_name = ENDPOINTS[endpoint]
        getattr(self.social_media_client, method_name)()

class GlobalSocialMediaUser(HttpUser):
    """new"""
    wait_time = between(1, 5)
    tasks = [GlobalUser]

```

Figure 4 - Test de charge locustfile_global.py

Question 2.3



The screenshot shows the Locust web interface. At the top, there's a green header with the Locust logo, host address (http://127.0.0.1:8080), status (RUNNING), users (10), RPS (3.6), and failures (0%). Below the header are tabs for STATISTICS, CHARTS, FAILURES, EXCEPTIONS, CURRENT RATIO, DOWNLOAD DATA, LOGS, and LOCUST CLOUD. The STATISTICS tab is active, displaying a table with the following data:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Create Post	23	0	2	2	2	1.82	1	2	213.61	0	0
POST	Follow User	57	0	1.05	2	3	1.52	1	3	40	0.5	0
POST	Like Post	110	0	2	2	2	1.51	1	2	37	1	0
POST	Login	10	0	3	3	3	2.76	2	3	55.5	0	0
GET	View Feed	185	0	2	2	4	1.61	1	9	3018.83	1	0
GET	View Profile	103	0	2	2	2	1.53	1	2	170.11	1.1	0
	Aggregated	488	0	2	2	3	1.6	1	9	1204.56	3.6	0

Figure 5 - Résultat de l'exécution de locustfile_global.py avec 10 utilisateurs

Partie 3:

Question 3.1

Le script d'extraction a été généré à l'aide de ChatGPT. Pour ce faire, nous avons copié une portion du fichier logs.txt contenant les différentes méthodes HTTP et demandé la génération d'un script capable de lire le fichier et d'en extraire les données nécessaires aux tests de charge basés sur une chaîne de Markov. Ce script récupère les séquences de requêtes par utilisateur afin de déterminer les transitions entre endpoints et de calculer les probabilités de transition.

Voici le résultat du script :

```
=== Chaîne de Markov (probabilités de transition) ===
/login          -> /login          : 1.0
/feed           -> /like/:id       : 0.673
/feed           -> /profile/:id    : 0.178
/feed           -> /post           : 0.149
/like/:id       -> /profile/:id    : 0.688
/like/:id       -> /feed           : 0.312
/profile/:id    -> /follow/:id     : 0.68
/profile/:id    -> /feed           : 0.32
/follow/:id     -> /feed           : 1.0
/post           -> /feed           : 1.0
```

Figure 6 - Résultat de l'exécution du script pour chaîne de Markov

Question 3.2

```

from locust import task, between, TaskSet, User, HttpUser
from extract_logs import markov_chain
from client import SocialMediaClient
import random

ENDPOINTS = {
    "/feed": "view_feed",
    "/like/:id": "like_post",
    "/profile/:id": "view_profile",
    "/follow/:id": "follow_user",
    "/post": "create_post"
}

# permet de déterminer quel endpoint arrive généralement après un endpoint
class MarkovUser(TaskSet):
    @osset(1)
    def on_start(self):
        self.social_media_client = SocialMediaClient(self.client)
        self.current_state = "/feed"

    @task
    @osset(1)
    def perform_api(self):
        if self.current_state in markov_chain:
            transitions = markov_chain[self.current_state]
            next_endpoint = random.choices(
                list(transitions.keys()),
                weights=list(transitions.values())
            )[0]

            if next_endpoint in ENDPOINTS:
                method_name = ENDPOINTS[next_endpoint]
                getattr(self.social_media_client, method_name)()

            self.current_state = next_endpoint

class MarkovSocialMediaUser(HttpUser):
    @osset(1)
    wait_time = between(1, 5)
    tasks = [MarkovUser]

```

Figure 7 - Test de charge locustfile_markov.py

Question 3.3

LOCUST

Host
http://127.0.0.1:8080

Status
RUNNING

Users
10

RPS
3.1

Failures
0%

EDIT

STOP

RESET



STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Create Post	13	0	2	3	3	1.95	1	3	211.31	0.3	0
POST	Follow User	42	0	2	2	3	1.58	1	3	40	0.2	0
POST	Like Post	70	0	1	2	2	1.47	1	2	37	0.9	0
POST	Login	10	0	3	3	3	2.87	2	3	55.5	0	0
GET	View Feed	88	0	2	2	3	1.8	1	3	15962.61	1.3	0
GET	View Profile	64	0	2	2	4	1.6	1	4	170.28	0.4	0
	Aggregated	287	0	2	2	3	1.69	1	4	4958.82	3.1	0



Figure 8 - Résultat de l'exécution de locustfile markov.py avec 10 utilisateurs

Partie 4:

Question 4.1

Nous avons augmenté progressivement le nombre d'utilisateurs (10 → 50 → 100 → 150 → etc...) avec un taux de montée de 10 utilisateurs/s. À chaque palier, le test a été exécuté environ une minute afin d'obtenir un volume suffisant de requêtes. Deux critères ont été utilisés pour évaluer la stabilité : le 95e centile, fixé à un seuil maximal de 500ms pour tous les types de requêtes, et un pourcentage d'erreurs qui doit être 0%.

Dans les deux scénarios (locustfile_global.py et locustfile_markov.py) les pourcentages d'erreurs sont restés excellents, toujours à 0%.

Par contre, les deux peuvent supporter 300 utilisateurs avant de dépasser le seuil de 500ms.

Question 4.2

Les tests ont été relancés avec la moitié de la charge maximale trouvée en Q4.1, soit 150 utilisateurs pour les deux scénarios respectivement : locustfile_global.py et locustfile_markov.py.

LOCUST

Host
http://127.0.0.1:8080

Status
RUNNING

Users
150

RPS
51

Failures
0%

EDIT

STOP

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Create Post	208	0	13	160	220	35.86	1	409	215.66	2.1	0
POST	Follow User	642	1	10	100	170	27.2	1	226	39.94	8	0
POST	Like Post	986	0	7	100	190	26.57	1	294	37	13	0
POST	Login	150	0	18	83	120	34.48	12	141	55.5	0	0
GET	View Feed	1388	2	40	140	200	55.86	1	296	1530536.29	18.1	0.1
GET	View Profile	876	0	9	96	190	25.81	2	263	171.87	9.8	0
Aggregated		4250	3	29	110	190	36.81	1	409	499917.7	51	0.1

LOCUST

Host
http://127.0.0.1:8080

Status
RUNNING

Users
150

RPS
49

Failures
0%

EDIT

STOP

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	Create Post	257	0	41	540	940	116.84	1	1114	217.99	2.8	0
POST	Follow User	636	0	45	340	630	87.23	1	746	40	7.8	0
POST	Like Post	1063	0	42	340	600	82.64	1	730	37	10.4	0
POST	Login	150	0	18	260	260	54.03	7	265	55.5	0	0
GET	View Feed	1512	0	92	460	650	143.1	39	761	2537651.52	16.7	0
GET	View Profile	982	0	45	380	650	92.98	2	765	173.07	11.3	0
Aggregated		4600	0	57	380	650	106.33	1	1114	834180.04	49	0

Figure 9 - Résultat de l'exécution des deux fichiers locustfile avec 150 utilisateurs

Voici les ratios sous forme de tableaux:

Tableau 1- Ratios pour locustfile_global.py (4250 requêtes):

Requête	Nombre de Requêtes	Ratio (%)
Create Post	208	4.9
Follow User	642	15.1
Like Post	986	23.2
Login	150	3.5
View Feed	1388	32.7
View Profile	876	20.6

Tableau 2- Ratios pour locustfile_markov.py (4600 requêtes):

Requête	Nombre de Requêtes	Ratio (%)
Create Post	257	5.59
Follow User	636	13.83
Like Post	1063	23.11
Login	150	3.26
View Feed	1512	32.9
View Profile	982	21.3

Les proportions observées dans ces deux cas sont très proches l'une de l'autre, et elles correspondent aussi aux ratios calculés dans la question 2.1. Cela indique que ces deux méthodes reproduisent fidèlement la charge réelle observée dans les logs.

Question 4.3

Les deux méthodes testées ont produit des résultats très similaires dans notre laboratoire : même capacité maximale (≈ 300 utilisateurs) et ratios très semblables. La charge globale reproduit uniquement les fréquences observées, tandis que la chaîne de Markov reproduit aussi l'enchaînement typique des actions. Dans notre cas, cet enchaînement n'a pas modifié les ratios ni la charge supportée, d'où la similitude des résultats.

La charge globale suffit donc lorsque seul le volume relatif des endpoints est important. La chaîne de Markov devient plus pertinente lorsque l'ordre des actions a un impact sur la charge réelle. Dans ce laboratoire, les deux méthodes ont pu être utilisées efficacement.